```dart
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:timezone/timezone.dart' as tz;
import 'package:timezone/data/latest.dart' as tz;
import 'package:shared_preferences/shared_preferences.dart';

// Firebase
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:lottie/lottie.dart';
import 'firebase_options.dart';

import 'settings.dart';
import 'profile.dart';
import 'login.dart';
import 'deal_activation.dart';
import 'notification_api.dart';

```

Om te beginnen gebruiken we altijd imports voor de onderdelen die we binnen de app willen gebruiken. Let hierbij altijd op de versies en voeg ze toe in het pubspec.yaml bestand

```
// DEEL 1: APP THEMA & ENTRY POINT

// Back  Type: String
@pragma('vm:entrypoint')
Future<void> firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
}

Run | Debug | Profile
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  FirebaseMessaging.onBackgroundMessage(firebaseMessagingBackgroundHandler);

  tz.initializeTimeZones();
  tz.setLocalLocation(tz.getLocation('Europe/Amsterdam'));

  await NotificationApi.init();

  String? token = await FirebaseMessaging.instance.getToken();
  print("FCM Token: $token");

  runApp(const AdventCalendarApp());
}

class AdventCalendarApp extends StatelessWidget {
  const AdventCalendarApp({super.key});

  @override
  Widget build(BuildContext context) {
    const Color darkGreen = Color(0xFF0C2521);
    const Color goldAccent = Color(0xFFEDBD35);
    const Color redAccent = Color(0xFFDC143C);

    return MaterialApp(
      title: 'MacA\'s Exclusieve Deals',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        scaffoldBackgroundColor: darkGreen,
        primaryColor: goldAccent,

        colorScheme: ColorScheme.fromSeed(
          seedColor: goldAccent,
          brightness: Brightness.dark,
          primary: goldAccent,
          secondary: redAccent,
          background: darkGreen,
        ), // ColorScheme.fromSeed

        appBarTheme: const AppBarTheme(
          color: Colors.transparent,
          elevation: 0,
          centerTitle: true,
          titleTextStyle: TextStyle(
            color: goldAccent,
            fontFamily: 'Georgia',
            fontSize: 24,
            fontWeight: FontWeight.w500,
            letterSpacing: 1.5,
          ), // TextStyle
          iconTheme: IconThemeData(color: goldAccent),
        ), // AppBarTheme

        textTheme: const TextTheme(
          bodyMedium: TextStyle(color: goldAccent, fontSize: 16),
        ), // TextTheme

        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: redAccent,
            foregroundColor: Colors.white,
            textStyle: const TextStyle(
              fontWeight: FontWeight.w600,
              fontSize: 16,
            ), // TextStyle
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(6),
            ), // RoundedRectangleBorder
          ),
        ), // ElevatedButtonThemeData

        bottomNavigationBarTheme: const BottomNavigationBarThemeData(
          selectedItemColor: goldAccent,
          unselectedItemColor: Color(0xFF3C5651),
          backgroundColor: darkGreen,
          elevation: 0,
          type: BottomNavigationBarType.fixed,
        ), // BottomNavigationBarThemeData
      ), // ThemeData
      home: const AuthWrapper(),
    ); // MaterialApp
  }
}
```

Hier zetten we eigenlijk alles vast zodat we het straks alleen aan te roepen.

- Initialiseert Firebase en pushmeldingen
- Stelt tijdzones en notificaties in.
- Haalt het FCM-token op.
- Start de Flutter-app met een custom thema.
- Bepaalt of de gebruiker naar de login- of hoofdscherm moet gaan.

```
class AuthWrapper extends StatelessWidget {
  const AuthWrapper({super.key});

  @override
  Widget build(BuildContext context) {
    final isLoggedIn = FirebaseAuth.instance.currentUser != null;
    return isLoggedIn ? const MainScreen() : const LoginScreen();
  }
}
```

Hier kijken we of de gebruiker is ingelog en dan het juiste scherm tonen zodat de gebruiker altijd een optie heeft om bijv. in te loggen, aanmelden of gelijk naar de homescreen gaat etc.

```
Future<void> _loginUser() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  final String email = _emailController.text.trim();
  final String password = _passwordController.text;

  setState(() {
    _isLoading = true;
  });

  try {
    await _auth.signInWithEmailAndPassword(email: email, password: password);

    if (mounted) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(const SnackBar(content: Text('Succesvol ingelogd!')));
```

Hier meldt een gebruiker zich aan met email + wachtwoord via FirebaseAuth.

```
// Functie voor Google Sign-In
Future<void> _signInWithGoogle() async {
  setState(() {
    _isLoading = true;
  });

  try {
    final GoogleSignInAccount? googleUser = await _googleSignIn.signIn();

    if (googleUser == null) {
      return; // Gebruiker heeft annuleren gekozen
    }

    final GoogleSignInAuthentication googleAuth =
        await googleUser.authentication;

    final AuthCredential credential = GoogleAuthProvider.credential(
      accessToken: googleAuth.accessToken,
      idToken: googleAuth.idToken,
    );

    UserCredential userCredential = await _auth.signInWithCredential(
      credential,
    );
    User? user = userCredential.user;

    if (user != null) {
      // Check of dit een nieuwe gebruiker is of een bestaande in Firestore
      DocumentSnapshot userDoc = await FirebaseFirestore.instance
          .collection('users')
          .doc(user.uid)
          .get();

      if (!userDoc.exists) {
        // Gebruiker bestaat nog niet in Firestore, sla basisgegevens op
        await FirebaseFirestore.instance
            .collection('users')
            .doc(user.uid)
            .set({
              'email': user.email,
              'displayName': user.displayName,
              'profilePictureUrl': user.photoURL,
              'registrationDate': FieldValue.serverTimestamp(),
            });
      }

      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Succesvol aangemeld met Google!')),
        );

        // Navigeer naar het hoofd scherm
        Navigator.pushAndRemoveUntil(
          context,
          MaterialPageRoute(builder: (context) => const CalendarScreen()),
          (Route<dynamic> route) => false,
        );
      }
    }
```

Deze klasse is het Login-scherm. Gebruikers kunnen inloggen via email/wachtwoord of Google. Bij succesvolle login stuurt de app ze door naar het hoofdscherm en worden nieuwe Google-gebruikers opgeslagen in Firestore. UI bevat validatie, laadindicatoren en navigatie naar het registratiescherm.

```
// Functie voor E-mail/Wachtwoord Registratie
Future<void> _registerUser() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  final String email = _emailController.text.trim();
  final String password = _passwordController.text;

  setState(() {
    _isLoading = true;
  });

  try {
    UserCredential userCredential = await _auth
        .createUserWithEmailAndPassword(email: email, password: password);
    User? user = userCredential.user;

    if (user != null) {
      // Sla basisgegevens op in Firestore
      await FirebaseFirestore.instance.collection('users').doc(user.uid).set({
        'email': email,
        'registrationDate': FieldValue.serverTimestamp(),
      });
    }
```

Dit scherm registreert een nieuw Firebase-account via email en wachtwoord, slaat basisgegevens op in Firestore en stuurt de gebruiker na succesvolle aanmelding door naar HomeScreen.

```dart
class _CalendarScreenState extends State<CalendarScreen> {
  static const int _calendarDays = 31;
  List<bool> _opened = List.filled(_calendarDays, false);
  String _userName = 'Exclusieve MacA Klant';
  int _dealsOpened = 0;
  bool _isLoadingUserData = true;

  Map<int, String> _firestoreDeals = {};

  //  Gegevens & Opslag Methoden

  Future<void> _fetchDeals() async {
    final snap = await FirebaseFirestore.instance.collection('deals').get();
    for (var doc in snap.docs) {
      int day = int.parse(doc.id);
      _firestoreDeals[day] = doc['text'];
    }
    if (mounted) setState(() {});
  }

  Future<void> _saveOpenedDayState() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setStringList(
      'openedDays',
      _opened.map((e) => e.toString()).toList(),
    );
    _dealsOpened = _opened.where((o) => o).length;
  }

  Future<void> _loadOpenedDayState() async {
    final prefs = await SharedPreferences.getInstance();

    final List<String>? savedOpenedDaysStrings = prefs.getStringList(
      'openedDays',
    );
    final List<bool>? savedOpenedDays = savedOpenedDaysStrings
        ?.map((e) => e == 'true')
        .toList();

    if (savedOpenedDays != null && savedOpenedDays.length == _calendarDays) {
      _opened = savedOpenedDays;
      _dealsOpened = _opened.where((o) => o).length;
    }
    if (mounted) setState(() {});
  }
```

```dart
class DealActivationScreen extends StatefulWidget {
  final String dealDescription;
  final int dealDay;

  const DealActivationScreen({
    required this.dealDescription,
    required this.dealDay,
    super.key,
  });

  @override
  State<DealActivationScreen> createState() => _DealActivationScreenState();
}

class _DealActivationScreenState extends State<DealActivationScreen> {

  int _countdownSeconds = 900;
  Timer? _timer;
  bool _isDealActive = true;

  @override
  void initState() {
    super.initState();
    _startTimer();
  }
```

```
// Zet de timer op 15 minuten
void _startTimer() {
  _timer = Timer.periodic(const Duration(seconds: 1), (timer) {
    if (!mounted) {
      timer.cancel(); // Stopt de timer als de widget niet meer in beeld is
      return;
    }
    setState(() {
      if (_countdownSeconds > 0) {
        _countdownSeconds--;
      } else {
        _timer?.cancel();
        _isDealActive = false; // Deal is verlopen
      }
    });
  }); // Timer.periodic
}

// Formatteert seconden naar M:SS
String _formatTime(int totalSeconds) {
  final minutes = totalSeconds ~/ 60;
  final seconds = totalSeconds % 60;
  return '${minutes.toString().padLeft(2, '0')}:${seconds.toString().padLeft(2, '0')}';
}

// Zorgt ervoor dat de timer stopt als het scherm gesloten wordt
@override
void dispose() {
  _timer?.cancel();
  super.dispose();
}
```

Het Kalender-scherm toont een adventskalender waarin deals per dag worden vrijgespeeld, lokaal worden opgeslagen en vanuit Firestore worden geladen, met animaties, notificatie-integratie en QR-code activatie.